

Helm Reader

Generated by Doxygen 1.9.4

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 Class Documentation	5
3.1 CourseFilter Class Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	6
3.1.2.1 begin()	6
3.1.2.2 getCogAvg()	6
3.1.2.3 getCourse()	6
3.1.2.4 getCourseAvg()	7
3.1.2.5 getCourseUpdate()	7
3.1.2.6 getHeadingAvg()	8
3.1.2.7 handle()	8
3.1.2.8 setBeta()	8
3.2 euler_angles_t Struct Reference	8
3.2.1 Detailed Description	9
3.2.2 Member Data Documentation	9
3.2.2.1 pitch	9
3.2.2.2 roll	9
3.2.2.3 yaw	9
3.3 FileManager Class Reference	9
3.3.1 Detailed Description	10
3.3.2 Constructor & Destructor Documentation	10
3.3.2.1 FileManager()	10
3.3.3 Member Function Documentation	11
3.3.3.1 available()	11
3.3.3.2 closeFile()	11
3.3.3.3 createDir()	11
3.3.3.4 deleteFile()	12
3.3.3.5 getFileManager()	12
3.3.3.6 listDir()	12
3.3.3.7 openFile()	13
3.3.3.8 removeDir()	13
3.3.3.9 renameFile()	13
3.3.4 Member Data Documentation	14
3.3.4.1 _debug	14
3.3.4.2 _fs	14
3.3.4.3 _mutex	14
3.3.4.4 _mutexSource	14

3.3.4.5 _outputStream	14
3.4 GPS Class Reference	15
3.4.1 Detailed Description	15
3.4.2 Constructor & Destructor Documentation	15
3.4.2.1 GPS()	15
3.4.2.2 ~GPS()	16
3.4.3 Member Function Documentation	16
3.4.3.1 begin()	16
3.4.3.2 fillReadings()	16
3.4.3.3 getCog()	16
3.4.3.4 getSog()	17
3.4.3.5 getTime()	17
3.4.3.6 handle()	17
3.4.3.7 pause()	18
3.4.3.8 resume()	18
3.5 IMU Class Reference	18
3.5.1 Detailed Description	19
3.5.2 Constructor & Destructor Documentation	19
3.5.2.1 IMU()	19
3.5.2.2 ~IMU()	19
3.5.3 Member Function Documentation	19
3.5.3.1 begin()	19
3.5.3.2 calibrate()	19
3.5.3.3 getFrequency()	20
3.5.3.4 getState()	20
3.5.3.5 getStateFromReadings()	20
3.5.3.6 initialized()	20
3.5.3.7 update()	21
3.6 Log Class Reference	21
3.6.1 Detailed Description	21
3.6.2 Constructor & Destructor Documentation	21
3.6.2.1 Log()	21
3.6.2.2 ~Log()	22
3.6.3 Member Function Documentation	22
3.6.3.1 begin()	22
3.7 MadgwickFilter Class Reference	22
3.7.1 Detailed Description	23
3.7.2 Member Function Documentation	23
3.7.2.1 geteuler_angles_t()	23
3.7.2.2 getquaternion_t()	23
3.7.2.3 initialize()	23
3.7.2.4 setGyroDrift()	24

3.7.2.5 update()	24
3.7.2.6 updateIMU()	25
3.7.3 Member Data Documentation	25
3.7.3.1 initialized	25
3.8 NMEA Class Reference	25
3.8.1 Detailed Description	26
3.8.2 Constructor & Destructor Documentation	26
3.8.2.1 NMEA()	26
3.8.2.2 ~NMEA()	27
3.8.3 Member Function Documentation	27
3.8.3.1 begin()	27
3.8.3.2 fillReadings()	27
3.8.3.3 getCog()	27
3.8.3.4 getHeading()	28
3.8.3.5 getRudderAngle()	28
3.8.3.6 getSog()	29
3.8.3.7 getTime()	29
3.8.3.8 getWindAngle()	29
3.8.3.9 handle()	30
3.8.3.10 pause()	30
3.8.3.11 resume()	30
3.9 OTA Class Reference	30
3.9.1 Detailed Description	31
3.9.2 Constructor & Destructor Documentation	31
3.9.2.1 OTA()	31
3.9.2.2 ~OTA()	31
3.9.3 Member Function Documentation	31
3.9.3.1 begin()	31
3.9.3.2 handle()	31
3.10 quaternion_t Struct Reference	32
3.10.1 Detailed Description	32
3.10.2 Member Data Documentation	32
3.10.2.1 w	32
3.10.2.2 x	32
3.10.2.3 y	32
3.10.2.4 z	33
3.11 RestApi Class Reference	33
3.11.1 Detailed Description	33
3.11.2 Member Function Documentation	33
3.11.2.1 begin()	33
3.12 SDManager Class Reference	34
3.12.1 Detailed Description	34

3.12.2 Constructor & Destructor Documentation	34
3.12.2.1 SDManager()	34
3.12.3 Member Function Documentation	34
3.12.3.1 available()	35
3.12.3.2 begin()	35
3.13 SPIFFSManager Class Reference	35
3.13.1 Detailed Description	36
3.13.2 Constructor & Destructor Documentation	36
3.13.2.1 SPIFFSManager()	36
3.13.3 Member Function Documentation	36
3.13.3.1 available()	36
3.13.3.2 begin()	36
3.14 wifi_settings_t Struct Reference	37
3.14.1 Detailed Description	37
3.14.2 Member Data Documentation	37
3.14.2.1 apPassword	37
3.14.2.2 apSsid	37
3.14.2.3 domain	38
3.14.2.4 password	38
3.14.2.5 ssid	38
3.15 WiFiManager Class Reference	38
3.15.1 Detailed Description	38
3.15.2 Constructor & Destructor Documentation	39
3.15.2.1 WiFiManager()	39
3.15.2.2 ~WiFiManager()	39
3.15.3 Member Function Documentation	39
3.15.3.1 begin()	39
3.15.3.2 handle()	39

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

CourseFilter	5
euler_angles_t	8
FileManager	9
SDManager	34
SPIFFSManager	35
GPS	15
IMU	18
Log	21
MadgwickFilter	22
NMEA	25
OTA	30
quaternion_t	32
RestApi	33
wifi_settings_t	37
WiFiManager	38

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CourseFilter	Class managing additional course filtering	5
euler_angles_t	Struct packaging rotation as Euler angles	8
FileManager	Class managing file access and file storage	9
GPS	Class managing built in GPS	15
IMU	Class managing built in IMU	18
Log	Class managing logging to SD card	21
MadgwickFilter	Class implementing Madgwick AHRS IMU filter	22
NMEA	Class managing NMEA bus sensors	25
OTA	Class managing Over The Air updates	30
quaternion_t	Struct packaging rotation as quaternion	32
RestApi	Class managing REST API	33
SDManager	Class managing SD card access	34
SPIFFSManager	Class managing SPIFFS access	35
wifi_settings_t	Struct packaging WiFi settings	37
WiFiManager	Class managing WiFi and AP of the device	38

Chapter 3

Class Documentation

3.1 CourseFilter Class Reference

Class managing additional course filtering.

```
#include <courseFilter.h>
```

Static Public Member Functions

- static void [begin](#) (float beta=0.5f, Stream *outputStream=&Serial)
Initialize course filtering.
- static void [handle](#) ()
Manually handle filtering if worker task not running.
- static bool [getCourse](#) (double &course)
Write current filtered course value to provided pointer.
- static bool [getCourseAvg](#) (double &courseAvg)
Write current average filtered course value to provided pointer.
- static bool [getCogAvg](#) (double &cogAvg)
Write current average COG value to provided pointer.
- static bool [getHeadingAvg](#) (double &headingAvg)
Write current average HDG value to provided pointer.
- static bool [getCourseUpdate](#) (double &course, double &courseAvg, double &cogAvg, double &headingAvg, double &lastCog, double &yawDiff)
Write updated derieved readings into provided pointers.
- static bool [setBeta](#) (float beta)
Set beta for course filtering.

3.1.1 Detailed Description

Class managing additional course filtering.

Class managing course filtering based on readings from different instruments. Basic filter is used after Madgwick AHRS to compensate gyro drift with COG from [GPS](#) instead unreliable Magnetometer. Available additional filters include moving average for COG, HDG and Madgwick+COG.

3.1.2 Member Function Documentation

3.1.2.1 begin()

```
void CourseFilter::begin (
    float beta = 0.5f,
    Stream * outputStream = &Serial ) [static]
```

Initialize course filtering.

Parameters

<i>beta</i>	How fast should value converge with COG reading
<i>outputStream</i>	Stream to write debug info to

3.1.2.2 getCogAvg()

```
bool CourseFilter::getCogAvg (
    double & cogAvg ) [static]
```

Write current average COG value to provided pointer.

Parameters

<i>cogAvg</i>	Average COG variable pointer
---------------	------------------------------

Returns

Value available & written

3.1.2.3 getCourse()

```
bool CourseFilter::getCourse (
    double & course ) [static]
```

Write current filtered course value to provided pointer.

Parameters

<i>course</i>	Course variable pointer
---------------	-------------------------

Returns

Value available & written

3.1.2.4 getCourseAvg()

```
bool CourseFilter::getCourseAvg (
    double & courseAvg ) [static]
```

Write current average filtered course value to provided pointer.

Parameters

<i>courseAvg</i>	Average course variable pointer
------------------	---------------------------------

Returns

Value available & written

3.1.2.5 getCourseUpdate()

```
bool CourseFilter::getCourseUpdate (
    double & course,
    double & courseAvg,
    double & cogAvg,
    double & headingAvg,
    double & lastCog,
    double & yawDiff ) [static]
```

Write updated derieved readings into provided pointers.

Parameters

<i>course</i>	Pointer to write filtered course
<i>courseAvg</i>	Pointer to write filtered course average
<i>cogAvg</i>	Pointer to write COG average
<i>headingAvg</i>	Pointer to write HDG average
<i>lastCog</i>	Pointer to write last COG reading
<i>yawDiff</i>	Pointer to write yaw delta

Returns

bool Course was updated since last call

3.1.2.6 getHeadingAvg()

```
bool CourseFilter::getHeadingAvg (
    double & headingAvg ) [static]
```

Write current average HDG value to provided pointer.

Parameters

<i>headingAvg</i>	Average heading variable pointer
-------------------	----------------------------------

Returns

Value available & written

3.1.2.7 handle()

```
void CourseFilter::handle ( ) [static]
```

Manually handle filtering if worker task not running.

3.1.2.8 setBeta()

```
bool CourseFilter::setBeta (
    float beta ) [static]
```

Set beta for course filtering.

Parameters

<i>beta</i>	
-------------	--

Returns

bool Beta set

The documentation for this class was generated from the following file:

- src/courseFilter.h

3.2 euler_angles_t Struct Reference

Struct packaging rotation as Euler angles.

```
#include <madgwick.h>
```

Public Attributes

- float [roll](#)
- float [pitch](#)
- float [yaw](#)

3.2.1 Detailed Description

Struct packaging rotation as Euler angles.

3.2.2 Member Data Documentation

3.2.2.1 pitch

```
float euler_angles_t::pitch
```

3.2.2.2 roll

```
float euler_angles_t::roll
```

3.2.2.3 yaw

```
float euler_angles_t::yaw
```

The documentation for this struct was generated from the following file:

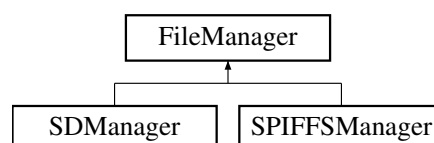
- `src/madgwick.h`

3.3 FileManager Class Reference

Class managing file access and file storage.

```
#include <FileManager.h>
```

Inheritance diagram for FileManager:



Public Member Functions

- [FileManager](#) (bool debug=false, Stream *outputStream=&Serial)
Construct a new File Manager:: File Manager object.
- bool [createDir](#) (const char *path)
Create directory in provided path.
- bool [removeDir](#) (const char *path)
Remove directory in provided path.
- fs::File [openFile](#) (const char *path, const char *mode=FILE_READ)
Open file and lock other file actions until file closed.
- void [closeFile](#) (fs::File &file)
Close file and unlock other actions.
- bool [renameFile](#) (const char *path1, const char *path2)
Rename file.
- bool [deleteFile](#) (const char *path)
Delete file in provided path.
- void [listDir](#) (const char *dirname, uint8_t levels=0)
List directory into output stream.

Static Public Member Functions

- static bool [available](#) ()
Check if file manager is available to use. Since this class is implementation to be inherited by real storage it is never available.
- static [FileManager](#) * [getFileManager](#) (bool debug=false, Stream *outputStream=&Serial)
Get most preferred storage [FileManager](#).

Protected Attributes

- SemaphoreHandle_t [_mutex](#)
- Stream * [_outputStream](#)
- bool [_debug](#)
- fs::FS * [_fs](#)

Static Protected Attributes

- static SemaphoreHandle_t [_mutexSource](#) {xSemaphoreCreateMutex()}

3.3.1 Detailed Description

Class managing file access and file storage.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 FileManager()

```
FileManager::FileManager (
    bool debug = false,
    Stream * outputStream = &Serial )
```

Construct a new File Manager:: File Manager object.

Parameters

<i>debug</i>	Print verbose operation logs to outputStream
<i>outputStream</i>	Stream to write debug info to

3.3.3 Member Function Documentation

3.3.3.1 available()

```
bool FileManager::available ( ) [static]
```

Check if file manager is available to use. Since this class is implementation to be inherited by real storage it is never available.

Returns

bool Storage available

3.3.3.2 closeFile()

```
void FileManager::closeFile (
    fs::File & file )
```

Close file and unlock other actions.

Parameters

<i>file</i>	Previously opened file
-------------	------------------------

3.3.3.3 createDir()

```
bool FileManager::createDir (
    const char * path )
```

Create directory in provided path.

Parameters

<i>path</i>	New directory path
-------------	--------------------

Returns

bool Directory created

3.3.3.4 deleteFile()

```
bool FileManager::deleteFile (
    const char * path )
```

Delete file in provided path.

Parameters

<i>path</i>	File path
-------------	-----------

Returns

bool File deleted

3.3.3.5 getFileManager()

```
FileManager * FileManager::getFileManager (
    bool debug = false,
    Stream * outputStream = &Serial ) [static]
```

Get most preferred storage [FileManager](#).

Parameters

<i>debug</i>	Print verbose operation logs to outputStream
<i>outputStream</i>	Stream to write debug info to

Returns

FileManager* Most preferred file manager

3.3.3.6 listDir()

```
void FileManager::listDir (
    const char * dirname,
    uint8_t levels = 0 )
```

List directory into output stream.

Parameters

<i>dirname</i>	Directory path
<i>levels</i>	Levels of recursion

3.3.3.7 openFile()

```
fs::File FileManager::openFile (
    const char * path,
    const char * mode = FILE_READ )
```

Open file and lock other file actions until file closed.

Parameters

<i>path</i>	File path
<i>mode</i>	Access mode

Returns

fs::File

3.3.3.8 removeDir()

```
bool FileManager::removeDir (
    const char * path )
```

Remove directory in provided path.

Parameters

<i>path</i>	Target directory path
-------------	-----------------------

Returns

bool Directory removed

3.3.3.9 renameFile()

```
bool FileManager::renameFile (
    const char * path1,
    const char * path2 )
```

Rename file.

Parameters

<i>path1</i>	Current path
<i>path2</i>	New path

Returns

bool File renamed

3.3.4 Member Data Documentation

3.3.4.1 `_debug`

```
bool FileManager::_debug [protected]
```

3.3.4.2 `_fs`

```
fs::FS* FileManager::_fs [protected]
```

3.3.4.3 `_mutex`

```
SemaphoreHandle_t FileManager::_mutex [protected]
```

3.3.4.4 `_mutexSource`

```
SemaphoreHandle_t FileManager::_mutexSource {xSemaphoreCreateMutex()} [static], [protected]
```

3.3.4.5 `_outputStream`

```
Stream* FileManager::_outputStream [protected]
```

The documentation for this class was generated from the following file:

- `src/fileManager.h`

3.4 GPS Class Reference

Class managing built in [GPS](#).

```
#include <gps.h>
```

Static Public Member Functions

- static void [begin](#) (Stream *outputStream=&Serial)
Initialize [GPS](#) communication.
- static void [pause](#) ()
Pause [GPS](#) worker task.
- static void [resume](#) ()
Resume [GPS](#) worker task.
- static void [handle](#) ()
Manually handle [GPS](#) communication if worker task not running.
- static bool [getCog](#) (double &_cog)
Write current COG value to provided pointer.
- static bool [getSog](#) (double &_sog)
Write current SOG value to provided pointer.
- static bool [getTime](#) (char &_time)
Write current time ISOSTRING value to provided pointer.
- static void [fillReadings](#) (JsonObject &readings)
Fill available readings into JSON under provided pointer.

Protected Member Functions

- [GPS](#) ()
- [~GPS](#) ()

3.4.1 Detailed Description

Class managing built in [GPS](#).

Class managing communication with [GPS](#) module connected by `Serial1` interface and parsing acquired data in NMEA0183 format

3.4.2 Constructor & Destructor Documentation

3.4.2.1 GPS()

```
GPS::GPS ( ) [inline], [protected]
```

3.4.2.2 ~GPS()

```
GPS::~~GPS ( ) [inline], [protected]
```

3.4.3 Member Function Documentation

3.4.3.1 begin()

```
void GPS::begin (
    Stream * outputStream = &Serial ) [static]
```

Initialize [GPS](#) communication.

Initialize communication interface for [GPS](#) module and create task handling it

Parameters

<i>outputStream</i>	Stream to write debug info to
---------------------	-------------------------------

3.4.3.2 fillReadings()

```
void GPS::fillReadings (
    JsonObject & readings ) [static]
```

Fill available readings into JSON under provided pointer.

Parameters

<i>readings</i>	Readings variable pointer
-----------------	---------------------------

3.4.3.3 getCog()

```
bool GPS::getCog (
    double & _cog ) [static]
```

Write current COG value to provided pointer.

Parameters

<i>_cog</i>	Course variable pointer
-------------	-------------------------

Returns

Value available & written

3.4.3.4 getSog()

```
bool GPS::getSog (
    double & _sog ) [static]
```

Write current SOG value to provided pointer.

Parameters

<code>_sog</code>	Speed variable pointer
-------------------	------------------------

Returns

Value available & written

3.4.3.5 getTime()

```
bool GPS::getTime (
    char & _time ) [static]
```

Write current time ISOSTRING value to provided pointer.

Parameters

<code>_time</code>	Time variable pointer
--------------------	-----------------------

Returns

Value available & written

3.4.3.6 handle()

```
void GPS::handle ( ) [static]
```

Manually handle [GPS](#) communication if worker task not running.

3.4.3.7 pause()

```
void GPS::pause ( ) [static]
```

Pause [GPS](#) worker task.

3.4.3.8 resume()

```
void GPS::resume ( ) [static]
```

Resume [GPS](#) worker task.

The documentation for this class was generated from the following file:

- [src/gps.h](#)

3.5 IMU Class Reference

Class managing built in [IMU](#).

```
#include <imu.h>
```

Static Public Member Functions

- static bool [begin](#) (Stream *outputStream=&Serial)
Initialize [IMU](#) communication & filtering.
- static bool [calibrate](#) ()
Calibrate [IMU](#) module.
- static bool [update](#) ()
Update [IMU](#) filter based on new readings if worker task not running.
- static [euler_angles_t](#) [getState](#) ()
Get current device rotation from filter.
- static [euler_angles_t](#) [getStateFromReadings](#) ()
Get current device rotation directly from sensor.
- static float [getFrequency](#) ()
Get filtering frequency.
- static bool [initialized](#) ()
Get initialized state.

Protected Member Functions

- [IMU](#) ()
- [~IMU](#) ()

3.5.1 Detailed Description

Class managing built in [IMU](#).

Class managing communication with Inertial Measurement Unit connected by `2wire` interface and filtering acquired data with Madgwick filter

3.5.2 Constructor & Destructor Documentation

3.5.2.1 IMU()

```
IMU::IMU ( ) [inline], [protected]
```

3.5.2.2 ~IMU()

```
IMU::~IMU ( ) [inline], [protected]
```

3.5.3 Member Function Documentation

3.5.3.1 begin()

```
bool IMU::begin (
    Stream * outputStream = &Serial ) [static]
```

Initialize [IMU](#) communication & filtering.

Initialize communication interface and filter for [IMU](#) module and create task handling them

Parameters

<i>outputStream</i>	Stream to write debug info to
---------------------	-------------------------------

3.5.3.2 calibrate()

```
bool IMU::calibrate ( ) [static]
```

Calibrate [IMU](#) module.

Calibrate [IMU](#) module and save calibration data to file on SD or SPIFFS. Device should not move during calibration

Returns

Calibration save success

3.5.3.3 getFrequency()

```
float IMU::getFrequency ( ) [static]
```

Get filtering frequency.

Returns

float Filtering frequency

3.5.3.4 getState()

```
euler_angles_t IMU::getState ( ) [static]
```

Get current device rotation from filter.

Returns

[euler_angles_t](#) Current device rotation in Euler angles

3.5.3.5 getStateFromReadings()

```
euler_angles_t IMU::getStateFromReadings ( ) [static]
```

Get current device rotation directly from sensor.

Returns

[euler_angles_t](#) Current device rotation in Euler angles

3.5.3.6 initialized()

```
bool IMU::initialized ( ) [static]
```

Get initialized state.

Returns

bool [IMU](#) initialized

3.5.3.7 update()

```
bool IMU::update ( ) [static]
```

Update [IMU](#) filter based on new readings if worker task not running.

Returns

bool Readings updated

The documentation for this class was generated from the following file:

- [src/imu.h](#)

3.6 Log Class Reference

Class managing logging to SD card.

```
#include <log.h>
```

Static Public Member Functions

- static void [begin](#) (Stream *outputStream=&Serial)
Initialize SD logging.

Protected Member Functions

- [Log](#) ()
- [~Log](#) ()

3.6.1 Detailed Description

Class managing logging to SD card.

Class managing logging sensor & filter data to SD card. Logging will be disabled if SD card not detected

3.6.2 Constructor & Destructor Documentation

3.6.2.1 Log()

```
Log::Log ( ) [protected]
```

3.6.2.2 ~Log()

```
Log::~~Log ( ) [protected]
```

3.6.3 Member Function Documentation

3.6.3.1 begin()

```
void Log::begin (
    Stream * outputStream = &Serial ) [static]
```

Initialize SD logging.

Initialize sd logging internals and create task handling it

Parameters

<i>outputStream</i>	Stream to write debug info to
---------------------	-------------------------------

The documentation for this class was generated from the following file:

- src/log.h

3.7 MadgwickFilter Class Reference

Class implementing Madgwick AHRS [IMU](#) filter.

```
#include <madgwick.h>
```

Public Member Functions

- void [initialize](#) ([euler_angles_t](#) angles, float proportionalGain)
Initialize filter with start angles.
- void [setGyroDrift](#) (float xd, float yd, float zd)
Set gyro drift to eliminate sensor drift during filtering.
- void [update](#) (float gx, float gy, float gz, float ax, float ay, float az, float mx, float my, float mz)
Update filter with values from [IMU](#) sensor (with magnetometer)
- void [updateIMU](#) (float gx, float gy, float gz, float ax, float ay, float az)
Update filter with values from [IMU](#) sensor (without magnetometer)
- [euler_angles_t](#) [geteuler_angles_t](#) ()
Get filtered reading in Euler angles.
- [quaternion_t](#) [getquaternion_t](#) ()
Get filtered reading in quaternion.

Public Attributes

- bool `initialized` = false

3.7.1 Detailed Description

Class implementing Madgwick AHRS [IMU](#) filter.

Class computing spatial rotation based on filtered sensors output Based on Sebastian Madgwick AHRS [IMU](#) integration algorithm: <https://x-io.co.uk/open-source-imu-and-ahrs-algorithms/>

3.7.2 Member Function Documentation

3.7.2.1 `geteuler_angles_t()`

```
euler_angles_t MadgwickFilter::geteuler_angles_t ( )
```

Get filtered reading in Euler angles.

Returns

`euler_angles_t` Filtered reading

3.7.2.2 `getquaternion_t()`

```
quaternion_t MadgwickFilter::getquaternion_t ( )
```

Get filtered reading in quaternion.

Returns

`quaternion_t` Filtered reading

3.7.2.3 `initialize()`

```
void MadgwickFilter::initialize (
    euler_angles_t angles,
    float proportionalGain = 0.1f )
```

Initialize filter with start angles.

Parameters

<i>angles</i>	Start angles
<i>proportionalGain</i>	Proportional gain of filtering

3.7.2.4 setGyroDrift()

```
void MadgwickFilter::setGyroDrift (
    float xd,
    float yd,
    float zd )
```

Set gyro drift to eliminate sensor drift during filtering.

Parameters

<i>xd</i>	X axis drift
<i>yd</i>	Y axis drift
<i>zd</i>	Z axis drift

3.7.2.5 update()

```
void MadgwickFilter::update (
    float gx,
    float gy,
    float gz,
    float ax,
    float ay,
    float az,
    float mx,
    float my,
    float mz )
```

Update filter with values from [IMU](#) sensor (with magnetometer)

Parameters

<i>gx</i>	Gyro X
<i>gy</i>	Gyro Y
<i>gz</i>	Gyro Z
<i>ax</i>	Accelerometer X
<i>ay</i>	Accelerometer Y
<i>az</i>	Accelerometer Z
<i>mx</i>	Magnetometer X
<i>my</i>	Magnetometer Y
<i>mz</i>	Magnetometer Z

3.7.2.6 updateIMU()

```
void MadgwickFilter::updateIMU (
    float gx,
    float gy,
    float gz,
    float ax,
    float ay,
    float az )
```

Update filter with values from [IMU](#) sensor (without magnetometer)

Parameters

<i>gx</i>	Gyro X
<i>gy</i>	Gyro Y
<i>gz</i>	Gyro Z
<i>ax</i>	Accelerometer X
<i>ay</i>	Accelerometer Y
<i>az</i>	Accelerometer Z

3.7.3 Member Data Documentation

3.7.3.1 initialized

```
bool MadgwickFilter::initialized = false
```

The documentation for this class was generated from the following file:

- `src/madgwick.h`

3.8 NMEA Class Reference

Class managing [NMEA](#) bus sensors.

```
#include <nmea.h>
```

Static Public Member Functions

- static bool [begin](#) (bool debug=false, bool forward=false, Stream *outputStream=&Serial)
Initialize [NMEA](#) bus communication.
- static void [pause](#) ()
Pause [NMEA](#) worker task.
- static void [resume](#) ()
Resume [NMEA](#) worker task.
- static void [handle](#) ()
Manually handle [NMEA](#) communication if worker task not running.
- static bool [getCog](#) (double &_cog)
Write current COG value to provided pointer.
- static bool [getHeading](#) (double &_heading)
Write current HDG value to provided pointer.
- static bool [getSog](#) (double &_sog)
Write current SOG value to provided pointer.
- static bool [getRudderAngle](#) (double &_rudderAngle)
Write current rudder angle value to provided pointer.
- static bool [getWindAngle](#) (double &_windAngle)
Write current wind angle value to provided pointer.
- static bool [getTime](#) (unsigned long &_time)
Write current time UNIX TIMESTAMP value to provided pointer.
- static void [fillReadings](#) (JsonObject &readings)
Fill available readings into JSON under provided pointer.

Protected Member Functions

- [NMEA](#) ()
- [~NMEA](#) ()

3.8.1 Detailed Description

Class managing [NMEA](#) bus sensors.

Class managing communication with [NMEA](#) bus connected by CAN interface and parsing acquired data in NMEA2000 format

3.8.2 Constructor & Destructor Documentation

3.8.2.1 NMEA()

```
NMEA::NMEA ( ) [inline], [protected]
```

3.8.2.2 ~NMEA()

```
NMEA::~NMEA ( ) [inline], [protected]
```

3.8.3 Member Function Documentation

3.8.3.1 begin()

```
bool NMEA::begin (
    bool debug = false,
    bool forward = false,
    Stream * outputStream = &Serial ) [static]
```

Initialize [NMEA](#) bus communication.

Initialize communication interface for [NMEA](#) bus and create task handling it

Parameters

<i>debug</i>	Write unrecognized NMEA PGN messages into outputStream
<i>forward</i>	Foreward incoming NMEA PGN messages into outputStream
<i>outputStream</i>	Stream to write debug info to

Returns

bool Initialized successfully

3.8.3.2 fillReadings()

```
void NMEA::fillReadings (
    JsonObject & readings ) [static]
```

Fill available readings into JSON under provided pointer.

Parameters

<i>readings</i>	Readings variable pointer
-----------------	---------------------------

3.8.3.3 getCog()

```
bool NMEA::getCog (
```

```
double & _cog ) [static]
```

Write current COG value to provided pointer.

Parameters

<code>_cog</code>	Course variable pointer
-------------------	-------------------------

Returns

Value available & written

3.8.3.4 getHeading()

```
bool NMEA::getHeading (
    double & _heading ) [static]
```

Write current HDG value to provided pointer.

Parameters

<code>_heading</code>	Heading variable pointer
-----------------------	--------------------------

Returns

Value available & written

3.8.3.5 getRudderAngle()

```
bool NMEA::getRudderAngle (
    double & _rudderAngle ) [static]
```

Write current rudder angle value to provided pointer.

Parameters

<code>_rudderAngle</code>	Ruddder angle variable pointer
---------------------------	--------------------------------

Returns

Value available & written

3.8.3.6 getSog()

```
bool NMEA::getSog (
    double & _sog ) [static]
```

Write current SOG value to provided pointer.

Parameters

<code>_sog</code>	Speed variable pointer
-------------------	------------------------

Returns

Value available & written

3.8.3.7 getTime()

```
bool NMEA::getTime (
    unsigned long & _time ) [static]
```

Write current time UNIX TIMESTAMP value to provided pointer.

Parameters

<code>_time</code>	Time variable pointer
--------------------	-----------------------

Returns

Value available & written

3.8.3.8 getWindAngle()

```
bool NMEA::getWindAngle (
    double & _windAngle ) [static]
```

Write current wind angle value to provided pointer.

Parameters

<code>_windAngle</code>	Wind angle variable pointer
-------------------------	-----------------------------

Returns

Value available & written

3.8.3.9 handle()

```
void NMEA::handle ( ) [static]
```

Manually handle [NMEA](#) communication if worker task not running.

3.8.3.10 pause()

```
void NMEA::pause ( ) [static]
```

Pause [NMEA](#) worker task.

3.8.3.11 resume()

```
void NMEA::resume ( ) [static]
```

Resume [NMEA](#) worker task.

The documentation for this class was generated from the following file:

- `src/nmea.h`

3.9 OTA Class Reference

Class managing Over The Air updates.

```
#include <ota.h>
```

Static Public Member Functions

- static void [begin](#) (const char *hostname, const char *password, Stream *outputStream=&Serial)
Initialize [OTA](#) handler and create task handling it.
- static void [handle](#) ()
Manually handle [OTA](#) updates if worker task not running.

Protected Member Functions

- [OTA](#) ()
- [~OTA](#) ()

3.9.1 Detailed Description

Class managing Over The Air updates.

3.9.2 Constructor & Destructor Documentation

3.9.2.1 OTA()

```
OTA::OTA ( ) [inline], [protected]
```

3.9.2.2 ~OTA()

```
OTA::~~OTA ( ) [inline], [protected]
```

3.9.3 Member Function Documentation

3.9.3.1 begin()

```
void OTA::begin (
    const char * hostname,
    const char * password,
    Stream * outputStream = &Serial ) [static]
```

Initialize [OTA](#) handler and create task handling it.

Parameters

<i>hostname</i>	
<i>password</i>	
<i>outputStream</i>	Stream to write debug info to

3.9.3.2 handle()

```
void OTA::handle ( ) [static]
```

Manually handle [OTA](#) updates if worker task not running.

The documentation for this class was generated from the following file:

- `src/ota.h`

3.10 quaternion_t Struct Reference

Struct packaging rotation as quaternion.

```
#include <madgwick.h>
```

Public Attributes

- float [w](#)
- float [x](#)
- float [y](#)
- float [z](#)

3.10.1 Detailed Description

Struct packaging rotation as quaternion.

3.10.2 Member Data Documentation

3.10.2.1 w

```
float quaternion_t::w
```

3.10.2.2 x

```
float quaternion_t::x
```

3.10.2.3 y

```
float quaternion_t::y
```

3.10.2.4 z

```
float quaternion_t::z
```

The documentation for this struct was generated from the following file:

- src/madgwick.h

3.11 RestApi Class Reference

Class managing REST API.

```
#include <api.h>
```

Static Public Member Functions

- static void [begin](#) (bool debug=false, Stream *outputStream=&Serial)
Initialize REST API.

3.11.1 Detailed Description

Class managing REST API.

3.11.2 Member Function Documentation

3.11.2.1 begin()

```
void RestApi::begin (
    bool debug = false,
    Stream * outputStream = &Serial ) [static]
```

Initialize REST API.

Parameters

<i>debug</i>	Write logs into outputStream
<i>outputStream</i>	Stream to write debug info to

The documentation for this class was generated from the following file:

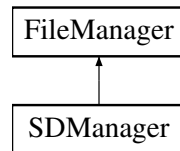
- src/api.h

3.12 SDManager Class Reference

Class managing SD card access.

```
#include <fileManager.h>
```

Inheritance diagram for SDManager:



Public Member Functions

- [SDManager](#) (bool debug=false, Stream *outputStream=&Serial)

Static Public Member Functions

- static bool [begin](#) (int detectPin=-1, Stream *outputStream=&Serial)
Initialize SD card communication & management.
- static bool [available](#) ()
Check if SD file manager is available to use.

Additional Inherited Members

3.12.1 Detailed Description

Class managing SD card access.

3.12.2 Constructor & Destructor Documentation

3.12.2.1 SDManager()

```
SDManager::SDManager (
    bool debug = false,
    Stream * outputStream = &Serial )
```

3.12.3 Member Function Documentation

3.12.3.1 available()

```
bool SDManager::available ( ) [static]
```

Check if SD file manager is available to use.

Returns

bool SD card available

3.12.3.2 begin()

```
bool SDManager::begin (
    int detectPin = -1,
    Stream * outputStream = &Serial ) [static]
```

Initialize SD card communication & management.

Parameters

<i>detectPin</i>	Pin for card detection (HIGH = card in slot)
<i>outputStream</i>	Stream to write debug info to

Returns

bool SD file manager initialized

The documentation for this class was generated from the following file:

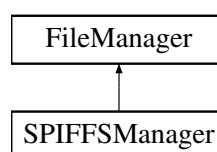
- src/fileManager.h

3.13 SPIFFSManager Class Reference

Class managing SPIFFS access.

```
#include <fileManager.h>
```

Inheritance diagram for SPIFFSManager:



Public Member Functions

- [SPIFFSManager](#) (bool debug=false, Stream *outputStream=&Serial)

Static Public Member Functions

- static bool [begin](#) (Stream *outputStream=&Serial)
Initialize SPIFFS communication & management.
- static bool [available](#) ()
Check if SPIFFS file manager is available to use.

Additional Inherited Members

3.13.1 Detailed Description

Class managing SPIFFS access.

3.13.2 Constructor & Destructor Documentation

3.13.2.1 SPIFFSManager()

```
SPIFFSManager::SPIFFSManager (
    bool debug = false,
    Stream * outputStream = &Serial )
```

3.13.3 Member Function Documentation

3.13.3.1 available()

```
bool SPIFFSManager::available ( ) [static]
```

Check if SPIFFS file manager is available to use.

Returns

bool SPIFFS available

3.13.3.2 begin()

```
bool SPIFFSManager::begin (
    Stream * outputStream = &Serial ) [static]
```

Initialize SPIFFS communication & management.

Parameters

<i>outputStream</i>	Stream to write debug info to
---------------------	-------------------------------

Returns

bool SPIFFS file manager initialized

The documentation for this class was generated from the following file:

- src/fileManager.h

3.14 wifi_settings_t Struct Reference

Struct packaging WiFi settings.

```
#include <wifiManager.h>
```

Public Attributes

- const char * [ssid](#) = nullptr
- const char * [password](#) = nullptr
- const char * [apSsid](#) = nullptr
- const char * [apPassword](#) = nullptr
- const char * [domain](#) = nullptr

3.14.1 Detailed Description

Struct packaging WiFi settings.

3.14.2 Member Data Documentation

3.14.2.1 apPassword

```
const char* wifi_settings_t::apPassword = nullptr
```

3.14.2.2 apSsid

```
const char* wifi_settings_t::apSsid = nullptr
```

3.14.2.3 domain

```
const char* wifi_settings_t::domain = nullptr
```

3.14.2.4 password

```
const char* wifi_settings_t::password = nullptr
```

3.14.2.5 ssid

```
const char* wifi_settings_t::ssid = nullptr
```

The documentation for this struct was generated from the following file:

- src/wifiManager.h

3.15 WiFiManager Class Reference

Class managing WiFi and AP of the device.

```
#include <wifiManager.h>
```

Static Public Member Functions

- static bool [begin](#) ([wifi_settings_t](#) settings, Stream *outputStream=&Serial)
Initialize WiFi and create task handling DNS if necessary.
- static void [handle](#) ()
Manually handle DNS if worker task not running.

Protected Member Functions

- [WiFiManager](#) ()
- [~WiFiManager](#) ()

3.15.1 Detailed Description

Class managing WiFi and AP of the device.

Class managing WiFi connection and Access Point capabilities of the device including onboard DNS

3.15.2 Constructor & Destructor Documentation

3.15.2.1 WiFiManager()

```
WiFiManager::WiFiManager ( ) [inline], [protected]
```

3.15.2.2 ~WiFiManager()

```
WiFiManager::~~WiFiManager ( ) [inline], [protected]
```

3.15.3 Member Function Documentation

3.15.3.1 begin()

```
bool WiFiManager::begin (
    wifi_settings_t settings,
    Stream * outputStream = &Serial ) [static]
```

Initialize WiFi and create task handling DNS if necessary.

Parameters

<i>settings</i>	WiFi settings to apply
<i>outputStream</i>	Stream to write debug info to

Returns

bool Properly initialized

3.15.3.2 handle()

```
void WiFiManager::handle ( ) [static]
```

Manually handle DNS if worker task not running.

The documentation for this class was generated from the following file:

- src/wifiManager.h

